

插件概述

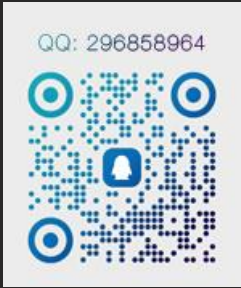
XGraph

XGraph 是一款基于 Unity Editor 的可视化节点编辑器插件，专为 **行为树 (Behavior Tree)**、**状态机 (State Machine)**、**任务流 (Task Flow)** 等图形化逻辑设计而开发。它采用 **GraphView** 作为核心框架，结合 **ScriptableObject 数据存储** 提供了一套完整的**节点编辑、连线、编组、撤销/重做、主题定制**等功能，适用于 **游戏 AI、任务系统、对话系统** 等复杂逻辑的可视化编辑。该插件具有 **模块化设计**，支持 **数据与视图分离**，并提供了 **可扩展的架构**，开发者可以轻松自定义节点类型、UI 样式以及交互逻辑

XGraph 不仅是工具，更是行为逻辑的设计框架
让你能够更专注于创意的实现以及工作流的简化

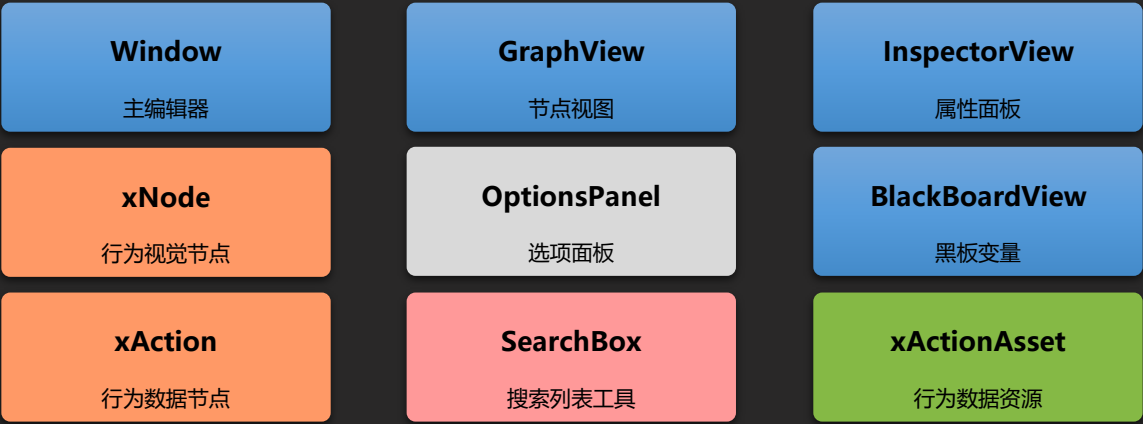
开源不易，您的支持是持续更新的动力，这个小工具倾注了我无数个深夜的调试与优化，它永远免费，但绝非无成本，如果您觉得这个工具能为您节省时间、解决问题，甚至带来一丝愉悦，请考虑赞助一杯咖啡，让我知道：有人在乎这份付出，而这将成为我熬夜修复Bug、添加新功能的最大动力。开源不是用爱发电，您的认可会让它走得更远

欢迎加入技术研讨群，在这里可以和我以及大家一起探讨插件的优化以及相关的技术实现思路，同时在做项目时遇到的众多问题以及瓶颈阻碍都可以互相探讨学习



Xgraph

核心构成 & 介绍



✔ 可视化节点编辑

- 节点创建与管理：支持多种节点类型（如行为节点、条件节点、中继节点、便签节点等）
- 连线系统：支持节点间的逻辑连接，并可通过 双击连线插入中继节点（Relay Node）来优化复杂连线
- 节点编组（Grouping）：可将多个节点归入编组，支持编组主题色定制，便于逻辑模块化管理
- 快捷键支持 Ctrl+D 克隆、Ctrl+A 全选、Ctrl+G 编组等
- 节点与数据解耦

✔ 数据持久化与资源管理

- ScriptableObject 存储：所有节点数据均以 .asset 文件存储，便于版本管理和团队协作
- 克隆机制（Clone Tree）：编辑时自动创建副本，避免直接修改原始资源，确保数据安全
- 自动保存视图状态（窗口尺寸、缩放、滚动位置等）

✔ 与Unity GraphToolKit的区别

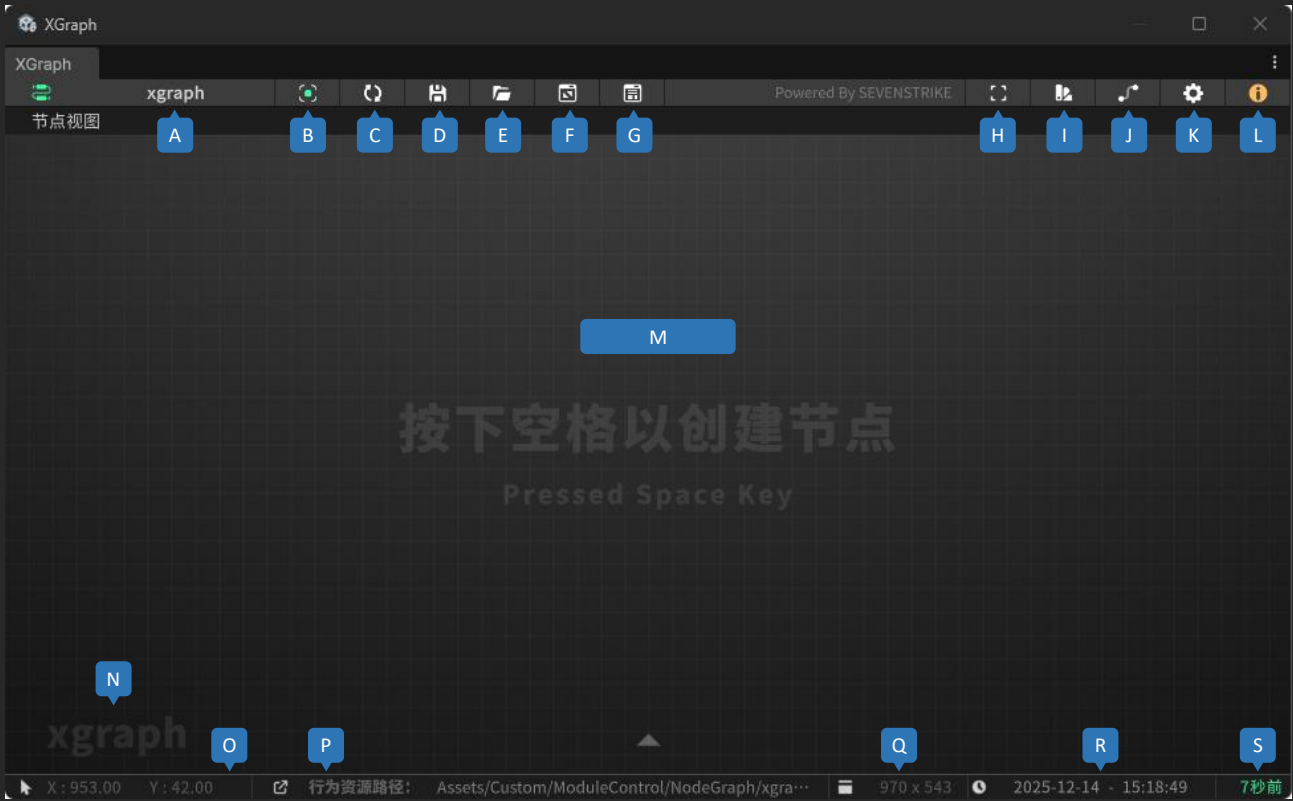
- UGT 像乐高万能积木，但拼特定城堡时你得先自己刻零件；XGraph是“直接给你城堡说明书”，换主题只要换贴纸
- 如果业务高度垂直且快速迭代，XGraph方案匹配度 > 90%
- UGT 的优势在跨领域复用与极致性能，但单点需求的落地速度远不及 XGraph

✔ 扩展性

- 自定义节点类型：通过继承 xAction_Base 和 xNode_Base 可轻松扩展新节点
- 自定义视觉节点：可对视觉节点进行最深度的行为扩展，包括属性面板的分组折叠菜单的扩展
- 事件回调：从编辑器本身到Graphview以及节点都提供相应的Action事件，便于扩展交互逻辑

Xgraph

GraphWindow节点编辑器

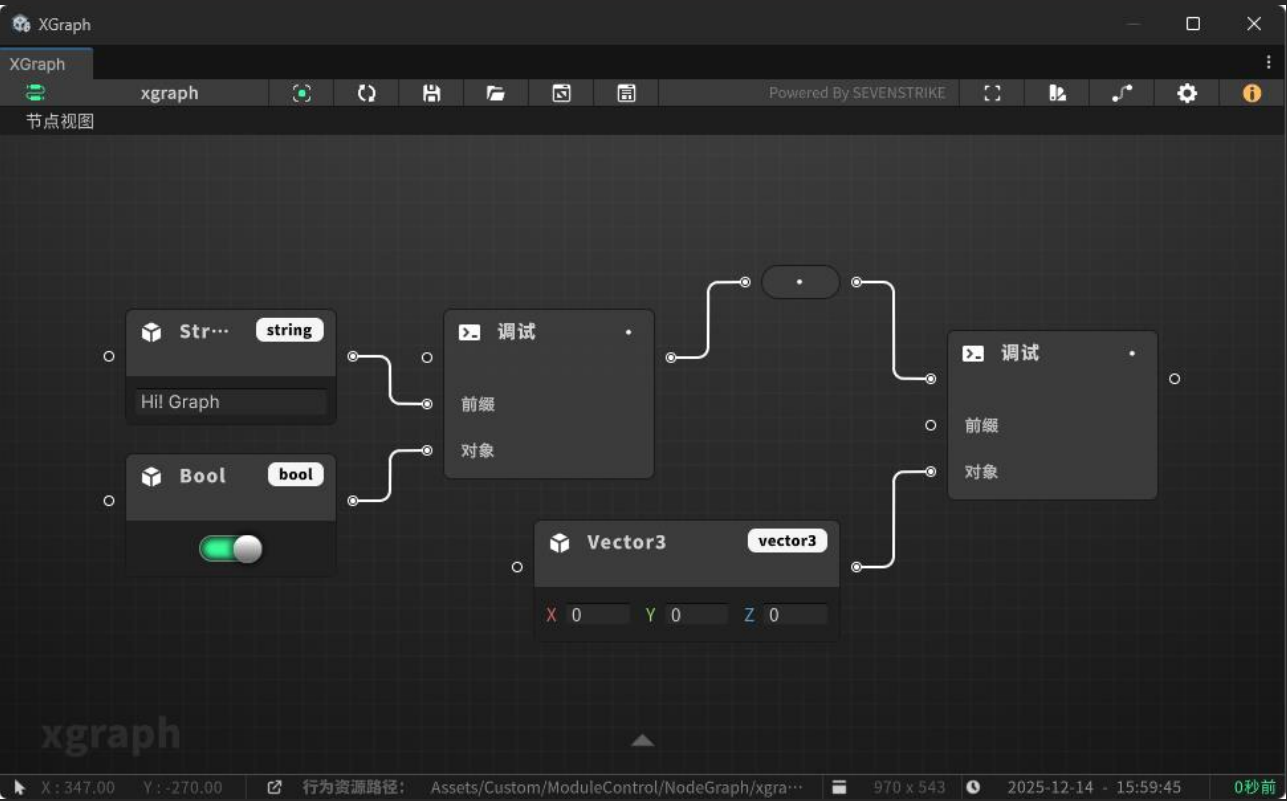


✅ 面板区域介绍

- **A:** 节点图工程名称
- **B:** 节点图快照
- **C:** 刷新当前工程
- **D:** 保存工程
- **E:** 打开工程
- **F:** 黑板变量面板开关
- **G:** 属性面板开关
- **H:** 聚焦
- **I:** 节点标记
- **J:** 数据流效果开关
- **K:** 编辑器选项面板开关
- **L:** 帮助手册链接
- **M:** 节点视图区域
- **N:** 工程名称水印
- **O:** 当前鼠标坐标
- **P:** 工程资源路径
- **Q:** 当前窗口尺寸
- **R:** 当前工程最后一次修改时间
- **S:** 当前工程距上一次的修改时间差

Xgraph

GraphView 节点



节点组成介绍

xAction_Base
行为数据类

xNode_Base
行为视觉类

xAction_Asset
行为视觉类

Xgraph

GraphView 节点

✔ xAction_Asset

- 该文件定义了xAction_Asset类（继承自ScriptableObject），是整个节点图系统的核心资源类，主要功能包括：
- 存储节点图编辑器的基础参数（如窗口尺寸、视图位置和缩放等）、主题配置（背景、选择框主题）以及功能开关状态（黑板、属性面板显示等）。
- 管理各类节点数据列表，包括行为节点（Actions）、变量节点（Variables）、便签（Sticks）、标签（Labels）、贴纸（Decals）、编组（Groups）和黑板变量（BlackboardVariable）。
- 提供节点的创建、克隆、移除，子节点关系管理（添加、获取子节点），变量值的获取与更新，以及快照操作（添加、覆盖、丢弃快照）等方法。
- 处理节点克隆时的子节点关系重建，确保克隆后的节点结构正确。
- 实现变量数据的刷新与同步，保证节点间变量引用的一致性

✔ xAction_Base

- 定义了xAction_Base类，是所有行为节点的基类，主要包含：
- 节点的基本属性（如名称identifyName、基础参数BaseArgs）。
- 丰富的回调事件，涵盖节点执行、主题色改变、变量 / 属性绑定与解绑、节点移动、尺寸改变等多种场景。
- 节点执行方法（Execute）、设置父节点（SetParentNode）、设置根资源（SetActionAssetRoot）等基础方法。
- 内部变量的绑定（InternalVariableData_Bind）与解绑相关逻辑，以及变量和属性数据的获取方法

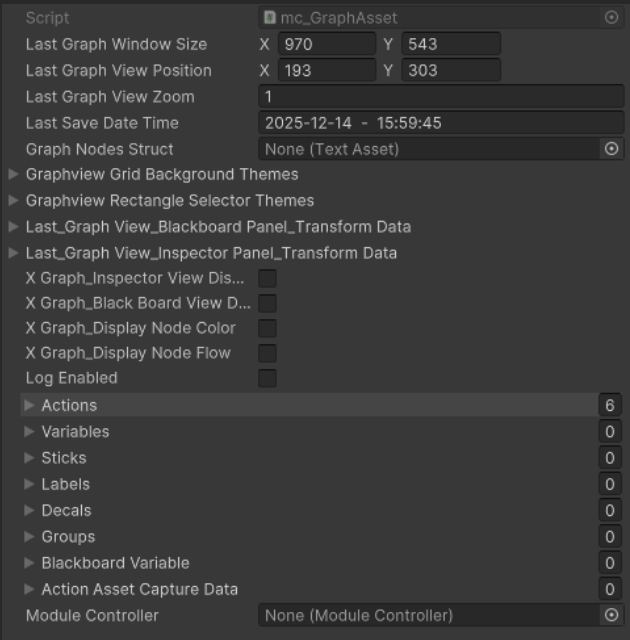
✔ xNode_Base

- xNode_Base 继承自 Unity 的 Node 类，是所有行为节点的基类，负责封装节点的通用功能，包括：
- 节点的初始化、绘制与样式管理
- 端口（输入 / 输出）的创建与连接
- 交互事件（选中、拖拽、尺寸变化等）的处理
- 数据绑定与同步（与底层行为数据 xAction_Base 关联）

Xgraph

xAction_Asset 类详解

xAction_Asset 是该节点可视化编辑工具的核心资源容器类，本质是继承自 Unity ScriptableObject 的可序列化资源文件，负责统一管理所有节点数据、编辑器配置、视图状态及快照等全局信息，是连接「可视化视图（xNode_* 类）」与「业务数据（xAction_Base 及其子类）」的核心枢纽。以下从核心定位、核心属性、核心方法、协作逻辑、关键特性五个维度展开详细介绍：



✔ 核心定位

- **资源载体**：作为独立的 Unity 资源文件 (.asset)，持久化存储整个节点图的所有数据（节点、变量、布局、样式），支持在项目中保存、复用、版本控制。
- **数据中枢**：集中管理所有 xAction_Base 子类实例（行为节点、变量节点），以及编辑器视图的配置信息，是视图层（xNode_）与数据层（xAction_）的数据同步核心。
- **功能管家**：封装节点图的全局操作（如节点克隆、快照、布局恢复、数据重载），提供统一的接口供编辑器视图调用。

✔ 核心属性（按功能分类）

xAction_Asset 的属性可分为「数据存储」「编辑器配置」「状态管理」三大类，均支持 Unity 序列化（可在 Inspector 面板编辑、持久化）：

xAction_Asset 是整个节点可视化工具的“数据大脑”，既承担了所有节点数据、编辑器配置的持久化存储职责，又提供了全局的节点管理、快照、同步接口，是连接视图层与数据层的核心桥梁。其设计核心是“统一管理、持久化、可回溯”，确保节点图的编辑状态可保存、可复用、可恢复，同时降低视图与数据的耦合度

分类	核心属性	作用说明
数据存储	List<xAction_Base> actions	存储所有行为节点数据（分支、调试、等待等 xAction_Base 子类实例）
	List<xAction_Variable> variables	存储所有变量节点数据（独立管理变量节点，便于黑板视图快速检索）
	List<xNode_GroupData> groups	存储节点编组数据（编组名称、位置、包含的节点 GUID、折叠状态）
	List<xAction_StickData> sticks	存储便签节点数据（文本、位置、尺寸、样式）
	List<xAction_LabelData> labels	存储标签节点数据（文本、字体、颜色、位置）
	Vector2 graphViewPos	节点画布视图的偏移位置（恢复编辑器时还原视图位置）
编辑器配置	float graphViewScale	节点画布的缩放比例（持久化视图缩放状态）
	EditorThemeConfig themeConfig	编辑器主题配置（背景色、网格色、选择框色、节点默认主题色）
	bool showBlackboard	黑板视图是否显示（记忆编辑器面板布局）
	bool showInspector	属性面板是否显示（记忆编辑器面板布局）
状态管理	List<ActionSnapshot> snapshots	节点图快照列表（存储不同版本的节点数据、布局，支持回溯）
	string sourceTreePath	源行为树资源路径（用于重载 / 同步外部行为树数据）
	Guid rootNodeGUID	根节点 GUID（标记节点图的入口节点，用于执行逻辑时定位起始点）

✔ 核心方法（按功能分类）

xAction_Asset 的方法围绕「节点管理」「数据同步」「视图恢复」「快照操作」四大核心场景设计，是编辑器交互的核心接口：

🔧 节点管理（增删改查 / 克隆）

方法名	核心逻辑
AddAction(xAction_Base action)	向 actions 列表添加节点数据，自动生成唯一 GUID，更新根节点关联（若为根节点）
RemoveAction(string guid)	根据 GUID 从 actions/variables 移除节点，同时删除关联的子节点 / 连线数据
GetActionByGUID(string guid)	根据 GUID 检索指定节点数据（返回 xAction_Base 实例，支持类型转换）
CloneAction(string sourceGUID)	克隆指定节点数据，生成新 GUID，自动处理子节点关联（递归克隆子节点）
RestructureGroups()	重建节点编组数据，清理无效编组（无节点的编组）、同步编组内节点 GUID

🔧 数据同步与重载

方法名	核心逻辑
SyncViewToAsset(xg_GraphView view)	将编辑器视图的布局（位置、缩放）、面板状态同步到 graphViewPos/showBlackboard 等属性
SyncAssetToView(xg_GraphView view)	从 graphViewPos/themeConfig 等属性恢复编辑器视图的布局、样式
ReloadTreeFromPath(string path)	根据 sourceTreePath 重载外部行为树数据，覆盖当前 actions/variables 列表
SaveAsset()	强制序列化资源，将内存中的节点数据、配置持久化到.asset 文件

 快照操作（版本回溯）

方法名	核心逻辑
Capture_Add(string name)	创建新快照：序列化当前所有节点数据、布局、配置，添加到 snapshots 列表
Capture_Covering(int index)	覆盖指定索引的快照：更新对应快照的节点数据、配置，保留快照名称
Capture_Load(int index)	加载指定快照：将快照中的数据还原到 actions/variables/ 布局属性中
CaptureClear()	清空所有快照，释放内存（保留最新的节点数据）

 辅助工具方法

方法名	核心逻辑
GetVariableThemeColor(Type varType)	根据变量类型（字符串 / 数字 / 布尔）从 themeConfig 获取对应的主题色
ValidateGUIDs()	校验所有节点 GUID 的唯一性，修复重复 GUID（避免数据冲突）
ResetEditorConfig()	重置编辑器配置为默认值（视图位置、面板显示状态、主题色）

Xgraph

xAction_Asset 类详解

✅ 与其他类的协作逻辑

xAction_Asset 是整个节点系统的 “数据中心”，与核心类的协作关系如下：

🔗 与 xAction_Base 及其子类

- **存储关系：**xAction_Asset 的 actions/variables 列表是所有 xAction_Base 子类实例的 “容器”，所有节点数据的增删改查均通过 xAction_Asset 提供的接口完成；
- **同步关系：**节点数据（如位置、主题色、参数）变更时，xAction_Base 会调用 xAction_Asset.SaveAsset() 触发持久化；xAction_Asset 加载快照时，会直接覆盖 xAction_Base 实例的属性。

🔗 与 xNode_Base 及其子类（视图层）

- **数据绑定：**编辑器初始化时，xg_GraphView 从 xAction_Asset 读取 actions/variables 数据，创建对应的 xNode_Base 子类实例，并调用 xNode_Base.BindData() 完成视图与数据的绑定；
- **交互同步：**用户操作 xNode_Base（如拖拽节点、编辑参数）时，视图类会调用 xAction_Asset 的 SyncViewToAsset()/AddAction() 等方法，将视图变更同步到数据层。

🔗 与 xg_GraphView（编辑器画布）

- **布局恢复：**xg_GraphView 启动时，调用 xAction_Asset.SyncAssetToView() 恢复上一次编辑的视图位置、缩放比例、面板状态；
- **全局操作：**xg_GraphView 提供的 “保存快照” “重载行为树” “重置布局” 等按钮，本质是调用 xAction_Asset 对应的快照 / 重载 / 重置方法

Xgraph

xAction_Asset 类详解

✓ 关键特性

🔗 持久化存储:

- 基于 ScriptableObject 实现，所有属性自动序列化，保存为独立的.asset 文件，支持 Unity 项目的版本控制（如 Git），可在不同场景 / 项目中复用节点图资源。

🔗 状态回溯:

- 通过快照功能（snapshots）保存节点图的不同版本，支持快速回滚到历史状态，便于调试不同逻辑方案

🔗 视图记忆:

- 持久化编辑器的布局（视图位置、缩放、面板显示状态），下次打开时自动恢复，提升编辑体验

🔗 数据校验:

- 内置 ValidateGUIDs() 等方法，避免节点 GUID 重复、无效编组、空引用等数据异常，保证节点图的完整性

🔗 外部联动:

- 通过 ReloadTreeFromPath() 支持从外部行为树资源同步数据，实现与其他系统（如游戏 AI 行为树）的联动

✓ 典型使用流程

🔗 创建资源:

- 在 Unity 编辑器中右键 → Create → xAction Asset，生成空的 xAction_Asset 资源文件

🔗 编辑节点:

- 在 xg_GraphView 中创建 / 编辑节点，视图层自动调用 xAction_Asset.AddAction()/SyncViewToAsset() 同步数据；

🔗 保存快照:

- 点击“保存快照”按钮，调用 Capture_Add() 存储当前节点图状态；

🔗 持久化:

- Unity 自动序列化 xAction_Asset，或手动调用 SaveAsset() 强制保存

🔗 重载 / 回溯:

- 如需恢复历史状态，调用 Capture_Load()；如需同步外部数据，调用 ReloadTreeFromPath()

Xgraph

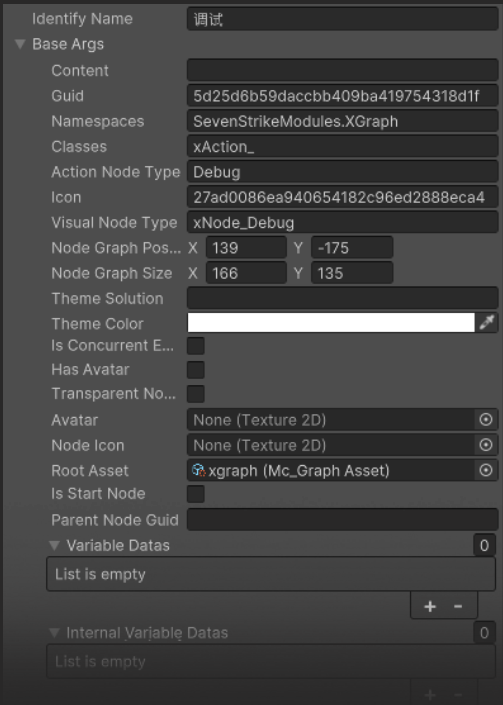
xAction_Base_ 类详解

xAction_Base 是整个节点可视化工具中行为类节点的核心基类，定义了行为节点的通用数据结构、执行逻辑和回调体系，所有 xAction_* 相关类均直接 / 间接继承自它，或与它形成数据绑定、功能扩展的关系。以下从继承关系、功能分工、数据交互三个维度详细拆解：

✅ 核心继承关系（父子类层级）

xAction_Base 作为根基类，衍生出多个专注于特定场景的子类

- xAction_Base（基类）
 - ├─ xAction_Variable（变量节点数据类）
 - ├─ xAction_Debug（调试节点数据类）
 - ├─ xAction_Branch（分支节点数据类，逻辑判断）
 - ├─ xAction_Composite（复合节点数据类，多子节点执行）
 - └─ xAction_Wait（等待节点数据类，延时/条件等待）



🔗 基类：xAction_Base

定位：所有行为节点的 “数据核心”，不对应具体功能，仅封装通用属性和方法。

核心属性：

- ✓ **基础标识：**identifyName（节点名称）、GUID（唯一标识）、position（节点位置）、size（节点尺寸）
- ✓ **逻辑关联：**parentGUID（父节点 GUID）、childGUIDs（子节点 GUID 列表）、portDatas（端口数据）
- ✓ **样式配置：**themeColor（主题色）、icon（节点图标）、markColor（标记色）
- ✓ **状态控制：**isEnabled（是否启用）、executeMode（执行模式：顺序 / 并发）

核心方法：

- ✓ **生命周期：**Initialize()（初始化）、Execute()（执行节点逻辑）、Reset()（重置状态）；
- ✓ **事件回调：**On_VariablesValue_Changed()（变量值变更回调）、On_ThemeColor_Changed()（主题色变更回调）；
- ✓ **数据管理：**BindVariable()（绑定变量）、UnbindVariable()（解绑变量）、SaveData()（保存数据）。

 子类：xAction_Variable (变量节点数据类)

定位：专门管理 “变量数据” 的节点，是变量节点（xNode_Variable）的底层数据载体

核心扩展：

- ✓ **新增属性：**variableType（变量类型：字符串 / 数字 / 布尔 / 向量等）、variableValue（变量值，泛型存储）、blackboardGUID（关联的黑板变量 GUID）；
- ✓ **新增方法：**Initialized()（根据变量类型初始化默认值）、SetValue()（设置变量值，自动触发回调）、GetValue()（获取变量值，支持类型转换）；

与基类的联动：

- ✓ 重写 On_VariablesValue_Changed()，当变量值修改时，同步更新所有关联的 xAction_Base 子类节点（如调试节点、分支节点）

 子类：xAction_Debug (调试节点数据类)

定位：用于 “调试 / 日志输出” 的节点，运行时输出自定义信息

核心扩展：

- ✓ **新增属性：**Prefix（日志前缀）、Msg（日志内容）、executeLog（是否输出执行日志）；
- ✓ **重写方法：**Execute()（运行时调用 Debug.Log() 输出 Prefix + Msg，支持绑定变量动态替换内容）；
- ✓ **扩展方法：**DebugMessage_withPort()（通过端口绑定变量，动态生成日志内容）、SetPrefix_withPort()（通过端口设置日志前缀）。

 其他子类（xAction_ / Branch / Composite / Wait)

子类	核心定位	关键扩展（对比基类）
xAction_Branch	条件分支逻辑（if-else）	新增 condition（判断条件）、trueGUID/falseGUID（分支节点 GUID）；重写 Execute() 执行条件判断并跳转对应分支
xAction_Composite	多子节点执行（顺序 / 并发）	新增 childExecuteOrder（子节点执行顺序）；重写 Execute() 按模式执行所有子节点
xAction_Wait	延时 / 条件等待	新增 waitTime（延时长）、waitCondition（等待条件）；重写 Execute() 触发等待逻辑

Xgraph

xAction_Base_ 类详解

✅ 非继承关系的关联类（功能扩展）

除直接子类外，还有部分 xAction_* 类不继承自 xAction_Base，但与它深度绑定，负责管理节点集合、资源存储等全局功能

🔗 xAction_Asset（节点图资源类）

定位：整个节点图的“资源容器”，存储所有 xAction_Base 子类实例的集合

核心关联逻辑：

- ✓ **存储属性：**actions (List<xAction_Base>，所有行为节点数据)、variables (List<xAction_Variable>，变量节点数据)、groups (节点编组数据)；
- ✓ **管理方法：**AddAction() (添加 xAction_Base 实例)、RemoveAction() (移除实例)、CloneAction() (克隆实例)、ReloadTree() (重载节点树)；
- ✓ **快照功能：**Capture_Add() (保存节点状态快照)、Capture_Covering() (覆盖快照)，本质是保存 xAction_Base 实例的属性快照。

🔗 xAction_Blackboard（黑板变量管理类）

定位：全局变量池，为所有 xAction_Base 子类提供变量共享能力

核心关联逻辑：

- ✓ **存储全局变量：**blackboardVariables (List<xAction_Variable>)
- ✓ **变量绑定：**BindVariableToAction() (将黑板变量绑定到指定 xAction_Base 节点)
- ✓ **同步更新：**当黑板变量值修改时，触发所有绑定的 xAction_Base 节点的 On_VariablesValue_Changed() 回调

✔ 数据交互与联动逻辑

所有 xAction_* 类最终通过 “数据 - 视图分离” 的模式协同工作，核心联动流程如下：

- ✔ **数据层 (xAction_类)**：xAction_Base 及其子类存储节点的 “业务数据”（逻辑、属性、关联关系），不涉及 UI 渲染；
- ✔ **视图层 (xNode_类)**：xNode_Base（节点视图类）绑定 xAction_Base 实例，将数据渲染为可视化节点（绘制 UI、端口、样式）；
- ✔ **交互层**：用户操作视图层（拖拽、编辑、连线）时，视图类调用 xAction_Base 的方法更新数据，数据变更后通过回调同步回视图层。

🔄 典型联动场景

- ✔ 编辑 xNode_Variable（变量节点视图）的数值 → 调用 xAction_Variable.SetValue() → 触发 On_VariablesValue_Changed() → 所有绑定该变量的 xAction_Debug/xAction_Branch 节点同步更新逻辑
- ✔ 执行 xNode_Branch（分支节点视图）→ 视图类调用 xAction_Branch.Execute() → 基类 xAction_Base 的 Execute() 执行通用逻辑 → 子类扩展逻辑（条件判断）→ 返回执行结果至视图层展示

简言之，xAction_Base 定义了 “行为节点该有什么通用能力”，而其他 xAction_* 类则回答 “具体场景下该怎么用这些能力” —— 基类保证了所有行为节点的一致性，子类则实现了功能的差异化扩展，最终通过 xAction_Asset 完成全局管理，形成 “基类定规则、子类做实现、容器做管理” 的完整体系

类名	与 xAction_Base 的关系	核心价值
xAction_Variable	直接子类	封装变量数据，为所有节点提供数据支撑
xAction_Debug	直接子类	扩展调试功能，可视化输出节点执行信息
xAction_Branch/Composite/Wait	直接子类	扩展特定业务逻辑（分支 / 复合 / 等待），复用基类执行体系
xAction_Asset	非继承，数据容器	管理所有 xAction_Base 实例，实现资源持久化
xAction_Blackboard	非继承，变量池	为 xAction_Base 子类提供全局变量绑定能力

Xgraph

xNode_Base_类详解

xNode_Base 是该节点可视化编辑工具中所有可视化节点视图的核心基类，定义了节点 UI 渲染、交互、端口管理的通用逻辑，所有 xNode_* 相关类均直接 / 间接继承自它，或与它形成 “功能扩展 / 配套协作” 的关系。以下从继承层级、功能分工、协作逻辑三个维度，详细拆解其与其他 xNode_* 类的关系和核心作用：

✔ 核心继承关系（视图类层级）

xNode_Base 作为视图层的 “骨架类”，衍生出适配不同业务节点的子类，层级结构清晰，核心脉络如下：



```
namespace SevenStrikeModules.XGraph
{
    using ...

    99+ 个引用 | SevenStrike, 4 天前 | 1 名作者, 14 项更改
    public class xNode_Base : Node
    {
        > 组件
        > 控件
        > 参数
        > 回调
        > 节点数据
        > 端口
        > 节点信息

        /// <summary> 初始化节点 - ActionNode_Base
        32 个引用 | SevenStrike, 9 天前 | 1 名作者, 3 项更改
        public virtual void Initialize(xg_GraphView graphView, Vector2 pos = default, xAction_Base data = null)

        /// <summary> 刷新节点标记主题颜色
        1 个引用 | SevenStrike, 9 天前 | 1 名作者, 2 项更改
        public void UpdateNodeThemeColor0...

        > 订阅 Graphview 克隆动作

        /// <summary> 当拖动节点位置时，将位置数据传递给对应的目标数据节点位置变量
        2 个引用 | SevenStrike, 4 天前 | 1 名作者, 5 项更改
        public override void SetPosition(Rect newPos)

        > 拖拽节点到节点
    }
}
```

🎯 基类：xNode_Base

定位：所有可视化节点的 “视图通用层”，封装节点 UI 绘制、交互、端口管理的核心逻辑，不对应具体业务功能，仅提供基础能力

核心属性（视图相关）：

- ✔ **容器与布局：**mainContainer（节点主容器）、titleContainer（标题容器）、portContainer（端口容器）；
- ✔ **交互状态：**isSelected（是否选中）、isHovered（是否悬停）、dragOffset（拖拽偏移）；
- ✔ **数据绑定：**actionData（绑定的底层数据类 xAction_Base）、graphView（所属的画布视图 xg_GraphView）；
- ✔ **视觉样式：**themeColor（主题色）、highlighter（高亮组件）、iconElement（图标元素）。

Xgraph

xNode_Base_ 类详解

核心方法（通用能力）：

- ✓ **UI 绘制**：Draw()（节点整体绘制）、DrawTitle()（标题绘制）、DrawPorts()（端口绘制）、UpdateStyle()（样式更新）；
- ✓ **交互处理**：OnDragStart()（拖拽开始）、OnDrag()（拖拽中）、OnSelect()（选中事件）、OnUnselect()（取消选中）；
- ✓ **数据同步**：BindData(xAction_Base data)（绑定底层数据）、SyncDataToView()（数据同步到视图）、SyncViewToData()（视图变更同步到数据）；
- ✓ **端口管理**：CreatePort()（创建端口）、RemovePort()（移除端口）、UpdatePortConnections()（更新端口连接）。

子类：xNode_Variable（变量节点视图）

定位：适配 xAction_Variable 数据类的可视化视图，专门渲染变量节点的 UI 和交互逻辑

核心扩展（对比基类）：

- ✓ **新增 UI 元素**：valueInputField（变量值编辑框）、typeLabel（变量类型标签）；
- ✓ **Draw()**：在基类基础上增加 “变量类型图标” “值编辑框” 的绘制；
- ✓ **SyncDataToView()**：从绑定的 xAction_Variable 同步变量类型、值到视图；
- ✓ **OnValueChanged()**：监听编辑框输入，将新值同步回 xAction_Variable；
- ✓ **交互扩展**：支持 “变量拖拽到画布” “变量与行为节点关联高亮” 等专属交互

子类：xNode_Label /xNode_Stick（辅助节点视图）

定位：轻量化辅助节点视图，无业务逻辑，仅用于文本说明 / 备注，是基类功能的极简扩展

核心扩展（对比基类）：

- ✓ **xNode_Label**：重写 Draw()，仅保留标题文本框，移除端口容器，支持文本字号、颜色自定义；
- ✓ **xNode_Stick**：新增 “背景透明度调节” “文本换行” 功能，重写 UpdateStyle() 适配便签式视觉样式，无端口、无数据绑定（仅存储文本和位置）。

 业务节点视图 (xNode_Branch/Composite/Wait/Debug)

这类子类是 xNode_Base 针对具体业务逻辑的视图适配，对应底层 xAction_* 数据类，核心扩展聚焦 “端口定制” 和 “业务 UI 渲染”，具体如下：

子类	绑定的数据类	核心扩展（对比 xNode_Base）
xNode_Branch	xAction_Branch	1. 定制端口：新增 “条件输入端口”“True/False 输出端口”； 2. 绘制扩展：添加 “条件表达式编辑框”； 3. 交互：条件变更时同步到 xAction_Branch
xNode_Composite	xAction_Composite	1. 定制端口：多输出端口（对应子节点执行顺序）； 2. 绘制扩展：显示 “执行模式（顺序 / 并发）” 标签； 3. 同步：子节点列表变更时更新端口数量
xNode_Wait	xAction_Wait	1. 绘制扩展：添加 “延时时长输入框”“等待条件选择框”； 2. 同步：时长 / 条件变更同步到 xAction_Wait
xNode_Debug	xAction_Debug	1. 绘制扩展：添加 “日志前缀输入框”“日志内容编辑框”； 2. 交互：支持绑定变量到日志内容，实时预览输出效果

Xgraph

xNode_Base_ 类详解

✅ 非继承关系的配套类（协作型 xNode_* 类）

除直接子类外，还有部分 xNode_* 类不继承自 xNode_Base，但作为“配套组件”与它深度协作，支撑节点系统的完整功能：

🔌 xNode_Port（节点端口类）

定位：节点端口的独立视图组件，由 xNode_Base 的 CreatePort() 方法创建，是节点间连接的核心载体

与 xNode_Base 的协作：

- ✓ **归属关系：**每个 xNode_Base 实例包含多个 xNode_Port（输入 / 输出），存储在 portList 属性中；
- ✓ **交互联动：**端口的点击、连线、断开事件，会通过 xNode_Base 同步到底层 xAction_Base 的端口数据；
- ✓ **样式同步：**xNode_Base 刷新主题色时，会同步更新所有所属 xNode_Port 的颜色。

🔌 xNode_Group（节点编组视图类）

定位：用于批量管理多个 xNode_Base 实例的编组容器，支持编组拖拽、折叠 / 展开

与 xNode_Base 的协作：

- ✓ **包含关系：**一个 xNode_Group 可包含多个 xNode_Base 实例，存储其引用；
- ✓ **位置同步：**编组拖拽时，批量更新内部所有 xNode_Base 的位置；
- ✓ **样式联动：**编组折叠时，隐藏内部 xNode_Base 的渲染，仅显示编组标题。

🔌 xNode_Decorator（节点装饰器视图类）

定位：附加在 xNode_Base 上的装饰组件（如“执行次数限制”“优先级标记”），无独立存在意义

与 xNode_Base 的协作：

- ✓ **挂载关系：**通过 AddDecorator() 方法挂载到 xNode_Base 的装饰器容器中；
- ✓ **数据联动：**装饰器的参数变更（如执行次数）同步到绑定的 xAction_Base 扩展属性中。

Xgraph

xNode_Base_ 类详解

✅ 核心协作逻辑（视图 - 数据 - 交互闭环）

所有 xNode_* 类最终围绕 xNode_Base 形成 “视图 - 数据 - 交互” 的完整闭环，核心流程如下：

- ✓ **初始化**：xg_GraphView（画布）从 xAction_Asset 加载 xAction_Base 数据 → 创建对应 xNode_Base 子类实例 → 调用 BindData() 绑定数据；
- ✓ **渲染**：xNode_Base 调用 SyncDataToView(), 从 xAction_Base 同步位置、样式、参数到 UI → 执行 Draw() 绘制节点及端口；
- ✓ **交互**：用户操作节点（拖拽、编辑值、连线）→ xNode_Base 触发交互方法（如 OnDrag()、OnValueChanged()）→ 调用 SyncViewToData() 将变更同步回 xAction_Base；
- ✓ **联动**：xAction_Base 数据变更后触发回调 → xNode_Base 接收回调并更新 UI（如主题色变化、端口连接变化）。

类名	与 xNode_Base 的关系	核心价值
xNode_Variable	直接子类	可视化渲染变量节点，适配变量编辑交互
xNode_Label/Stick	直接子类	轻量化辅助节点，简化基类逻辑实现文本备注
xNode_Branch/Debug 等	直接子类	适配具体业务节点，定制端口和业务 UI
xNode_Port	配套组件（组合）	实现节点间连接，是基类端口能力的载体
xNode_Group	配套组件（管理）	批量管理多个 xNode_Base 实例，提升编辑效率
xNode_Decorator	配套组件（附加）	扩展基类功能，添加装饰性业务逻辑

简言之，xNode_Base 定义了 “可视化节点该有什么通用视图能力”，子类则根据业务场景 “定制专属 UI 和交互”，配套类则从 “端口、编组、装饰” 等维度补充能力，最终共同构建了一套 “通用化基类 + 差异化子类 + 配套组件” 的节点视图体系，实现了从 “纯数据” 到 “可视化交互” 的完整落地

☒ 节点视图的背景主题选项

配色

背景色

变量面板背景

网格

属性面板背景

分界线

背景渐变反转

背景图像着色

主题色

主题样式方案

重置

☒ 节点视图的网格参数

参数

网格间距

18

分界线数量

18

背景图像

None

恢复默认

☒ 节点视图选择框参数

选框

线框色

线分段

4

坐标显示

恢复默认

☒ 黑板变量类型高亮筛选

标记

String

Color

Bool

Vector 2

Float

Vector 3

Integer

Vector 4



xg_BlackBoardView 类详细分析与说明

xg_BlackBoardView 是节点可视化编辑工具中黑板视图（变量面板）的核心实现类，本质是 Unity Editor 下的可视化面板组件（继承自 VisualElement），专门用于集中管理、展示和编辑全局 / 局部变量（基于 class_Variable 封装），是变量节点（xNode_Variable）与底层数据（xAction_Variable/class_Variable）的可视化操作入口，也是整个工具中“变量管理”的核心交互载体。通过可视化面板整合了变量的创建、编辑、展示、联动等全流程能力，既是变量数据的“可视化入口”，也是变量与节点之间的“联动桥梁”。其设计核心是“集中管理、高效交互、数据一致”，最终降低变量管理的复杂度，提升整个节点编辑流程的效率，是工具中不可或缺的核心面板之一

✔ 核心定位与设计目标

🎯 核心定位

- **变量总览面板：**展示所有全局黑板变量、节点关联变量，支持快速检索和筛选；
- **变量操作入口：**提供变量的创建、编辑、删除、重命名、类型修改等核心操作；
- **数据联动桥梁：**实现 “黑板变量” 与 “节点变量” 的双向绑定，变量值变更时自动同步到所有关联节点；
- **样式定制载体：**基于 class_BlackBoardViewTheme 实现面板样式的个性化配置与持久化。

🎯 设计目标

- **降低变量管理成本：**集中管理所有变量，避免分散在各个节点中难以维护；
- **提升交互效率：**支持拖拽变量到画布生成 xNode_Variable 节点，减少手动创建节点的操作；
- **保证数据一致性：**变量属性 / 值变更时，自动同步到所有关联的 xAction_Base 子类节点（如 xAction_Debug/xAction_Branch）。

✔ 核心结构（属性 + 组件）

xg_BlackBoardView 的核心结构围绕 “UI 组件” “数据关联” “样式配置” 三大维度设计，关键属性如下：

类别	核心属性 / 组件	作用说明
UI 核心组件	rootContainer	面板根容器，承载所有子组件（标题栏、搜索框、变量列表、操作按钮）
	searchField	变量搜索输入框，支持按名称 / 类型筛选变量
	variableListContainer	变量列表容器，每个子项为 VariableItemElement（单个变量的可视化项）
	createVariableButton	创建新变量的按钮，点击弹出“变量类型选择菜单”
	refreshButton	刷新变量列表按钮，同步 xAction_Asset 中的最新变量数据
数据关联	actionAsset	绑定的 xAction_Asset 实例，读取 / 写入变量数据（variables 列表）
	graphView	关联的画布视图 xg_GraphView，支持拖拽变量到画布生成 xNode_Variable
	variableItemMap	变量 GUID 与 VariableItemElement 的映射字典，快速定位变量可视化项
样式配置	theme	面板专属样式配置（class_BlackBoardViewTheme 实例），控制背景、字体、配色
	isFold	面板折叠状态，配合 class_ElementTransformData 实现布局记忆
状态控制	selectedVariableGUID	当前选中的变量 GUID，用于同步到属性面板（xg_InspectorView）展示详情
	filterType	变量筛选类型（枚举：全部 / 字符串 / 数字 / 布尔 / 向量等）

🔗 关键子组件：VariableItemElement

每个变量在列表中对应该一个 VariableItemElement（xg_BlackBoardView 内部嵌套类），核心功能：

- **展示变量核心信息：**名称、类型（图标）、当前值、是否全局；
- **支持交互操作：**双击重命名、右键菜单（删除 / 复制 GUID / 修改类型）、拖拽到画布生成节点；
- **状态反馈：**选中时高亮（匹配 class_BlackBoardViewTheme.variableItemHoverColor）、值变更时闪烁提示。

Xgraph

BlackBoardView 黑板变量

✅ 核心功能与关键方法

xg_BlackBoardView 的核心功能围绕 “变量生命周期管理” “交互联动” “样式同步” 展开，关键方法及逻辑如下：

🔗 初始化与样式加载

核心方法：Initialize(xAction_Asset asset, xg_GraphView graphView)

与 xNode_Base 的协作：

- ✓ 绑定 xAction_Asset 和 xg_GraphView，读取 asset 中的 variables 列表（xAction_Variable 实例）；
- ✓ 加载 class_BlackBoardViewTheme 样式配置（从 asset.themeConfig.blackBoardTheme 读取），初始化面板背景、字体、边距等；
- ✓ 注册组件事件：搜索框输入事件、创建按钮点击事件、变量列表项选中 / 拖拽事件；
- ✓ 调用 RefreshVariableList() 渲染初始变量列表。

🔗 变量列表管理

(1) 刷新列表：RefreshVariableList()与 xNode_Base 的协作：

- ✓ 清空 variableListContainer，遍历 actionAsset.variables 列表；
- ✓ 对每个 xAction_Variable，提取底层 class_Variable 数据，创建 VariableItemElement 并添加到列表；
- ✓ 应用筛选条件（searchField 输入内容 + filterType），隐藏不匹配的变量项；
- ✓ 更新 variableItemMap 映射关系，保证 GUID 与可视化项一一对应。

(2) 筛选变量：OnSearchFieldChanged(string searchText)/OnFilterTypeChanged(VariableType type)

- 实时过滤 variableListContainer 中的 VariableItemElement；
- 按名称：匹配 searchText 模糊查询变量名；
- 按类型：筛选 class_Variable.type 与 filterType 一致的变量；

Xgraph

BlackBoardView 黑板变量

✅ 核心交互：拖拽生成节点

关键逻辑：RegisterDragAndDropEvent()

- 为 VariableItemElement 注册拖拽事件 (DragStartEvent/DragPerformEvent) ;
- 拖拽开始时：封装变量 GUID / 类型 / 值为拖拽数据 (DragAndDrop.SetData()) ;
- 拖拽完成 (画布释放) 时：xg_GraphView 捕获拖拽数据，创建 xNode_Variable 节点并绑定对应的 xAction_Variable;
- 自动定位：生成的节点位置为鼠标释放位置，对齐画布网格。

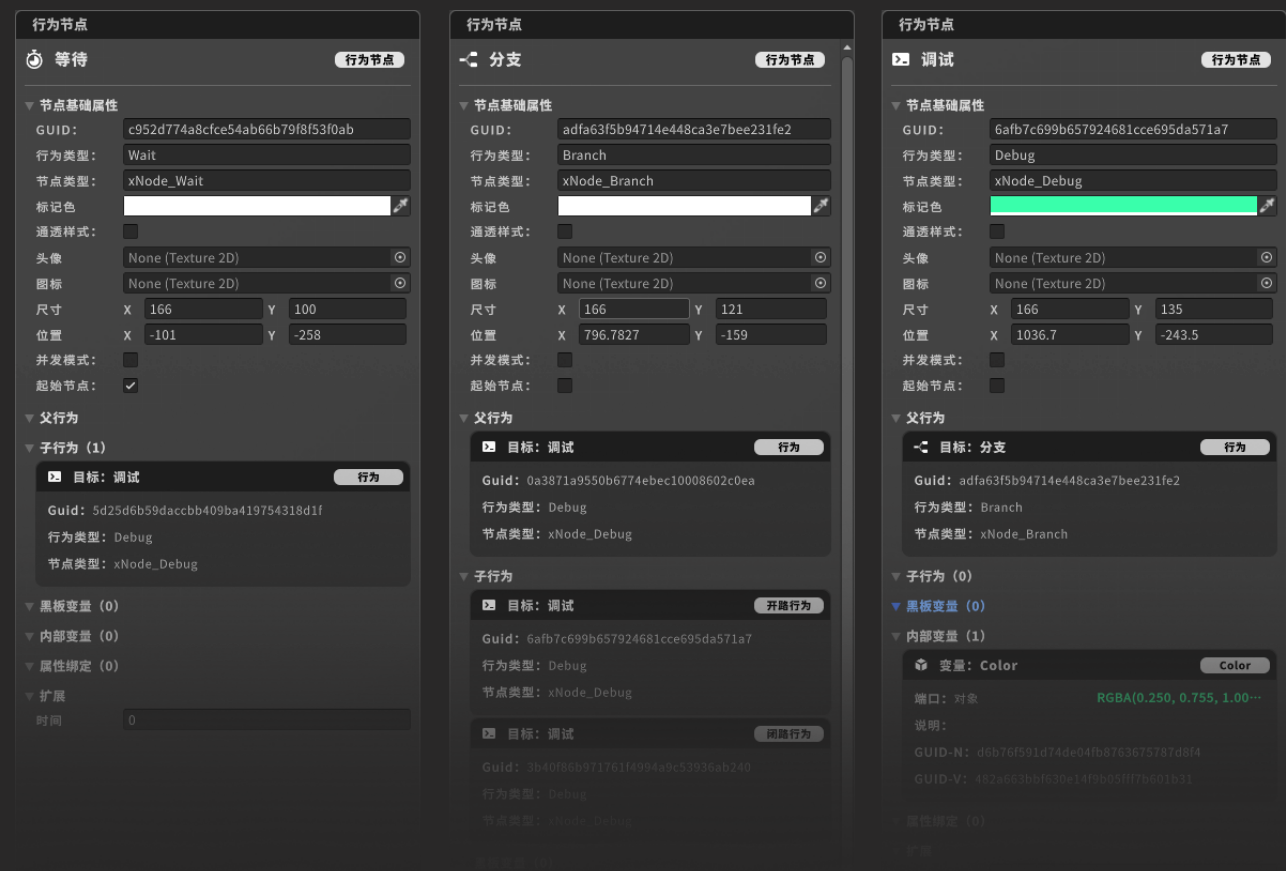
✅ 数据同步与事件监听

xg_BlackBoardView 通过事件监听实现 “数据 - 视图” 双向同步，核心监听逻辑：

- 监听 xAction_Asset 变量变更事件：当 actionAsset.variables 列表增删改时，自动调用 RefreshVariableList();
- 监听 class_Variable.OnValueChanged 事件：变量值变更时，更新对应 VariableItemElement 的显示值;
- 监听 xg_InspectorView 变量编辑事件：属性面板修改变量属性后，同步刷新黑板视图的变量项;
- 监听面板布局变更事件：面板折叠 / 展开 / 尺寸调整时，将布局数据 (class_ElementTransformData) 同步到 xAction_Asset 持久化。

✅ 核心特性与优势

- **集中化管理**：所有变量统一展示在面板中，支持搜索 / 筛选，解决变量分散管理的痛点;
- **拖拽式交互**：拖拽变量生成节点，大幅降低节点创建的操作成本;
- **数据一致性**：变量变更自动同步到所有关联节点，避免手动修改多个节点的变量值;
- **样式可定制**：基于主题类实现面板样式个性化，适配不同用户的操作习惯;
- **布局记忆**：面板折叠 / 尺寸 / 位置随 xAction_Asset 持久化，提升编辑体验;
- **引用校验**：删除变量时校验节点引用，避免误删导致的节点逻辑异常。



xg_InspectorView 类详细分析与说明

xg_InspectorView 是节点可视化编辑工具中的属性检视面板（继承自 Unity VisualElement），核心作用是实时展示并编辑编辑器中选中对象（节点、变量、编组等）的详细属性，是用户精细化配置对象参数的核心交互入口。

核心特性

- **多类型对象适配**：支持展示 / 编辑 xNode_Base 子类（行为节点、变量节点）、class_Variable（变量）、xNode_Group（编组）等不同类型对象的属性；
- **属性实时同步**：选中对象变更时自动刷新面板内容，编辑属性后立即同步到底层数据（如修改节点位置 / 名称同步至 xAction_Base，修改变量值同步至 class_Variable）；
- **结构化展示**：按 “基础属性（名称、GUID、位置）、样式属性（主题色、图标）、业务属性（节点执行模式、变量类型）” 分类展示，支持折叠 / 展开；
- **布局记忆**：面板的折叠状态、尺寸、位置通过 class_ElementTransformData 存储至 xAction_Asset，下次打开自动恢复；
- **校验与提示**：编辑属性时做合法性校验（如变量类型转换、GUID 唯一性检查），异常时给出可视化提示。。